Объектно-ориентированное программирование

6 семестр
Лекция 7.

Содержание лекции:

- Валидация модели на стороне сервера
- Валидация модели на стороне клиента
- Entity Framework

Преподаватель курса: Клюкин Даниил Анатольевич, ст. преподаватель каф. ПМиИТ

Валидация модели на стороне сервера

Важную роль в ASP.NET Core MVC играет валидация входных данных. Валидация позволяет проверить входные данные на наличие неправильных, корректных значений и должным образом обработать эти значения. Валидация модели в ASP.NET Core MVC базируется на общем механизме валидации, который имеется в .NET, однако ASP.NET Core MVC добавляет некоторую дополнительную инфраструктуру, которая облегчает процесс валидации.

Для рассмотрения валидации возьмем самый простейший проект ASP.NET Core по типу по типу **ASP.NET Core Empty**.

Вначале добавим в проект новую папку Models для моделей и определим в ней одну единственную модель - класс Person:

```
public class Person
{
    [Required]
    public string? Name { get; set; }
    [Required]
    public int Age { get; set; }
}
```

Здесь каждое свойство модели помечено атрибутом **Required**, который находится в пространстве имен **System.ComponentModel.DataAnnotations**. Данный атрибут указывает, что этим свойствам необходимо обязательно передать некоторые значения.

Затем определим в проекте папку **Controllers** для контроллеров приложения и в нее добавим контроллер **HomeController**. В контроллере HomeController действие Create, через которое мы будем добавлять на сервер объект модели Person:

```
public class HomeController : Controller
{
    public IActionResult Create() => View();

    [HttpPost]
    public string Create(Person person)
    {
        if (ModelState.IsValid)
            return $"{person.Name} - {person.Age}";
        return "Данные не прошли валидацию";
    }
}
```

Первый метод Create будет обрабатывать Get-запросы и будет отдавать пользователю представление с формой для добавления данных.

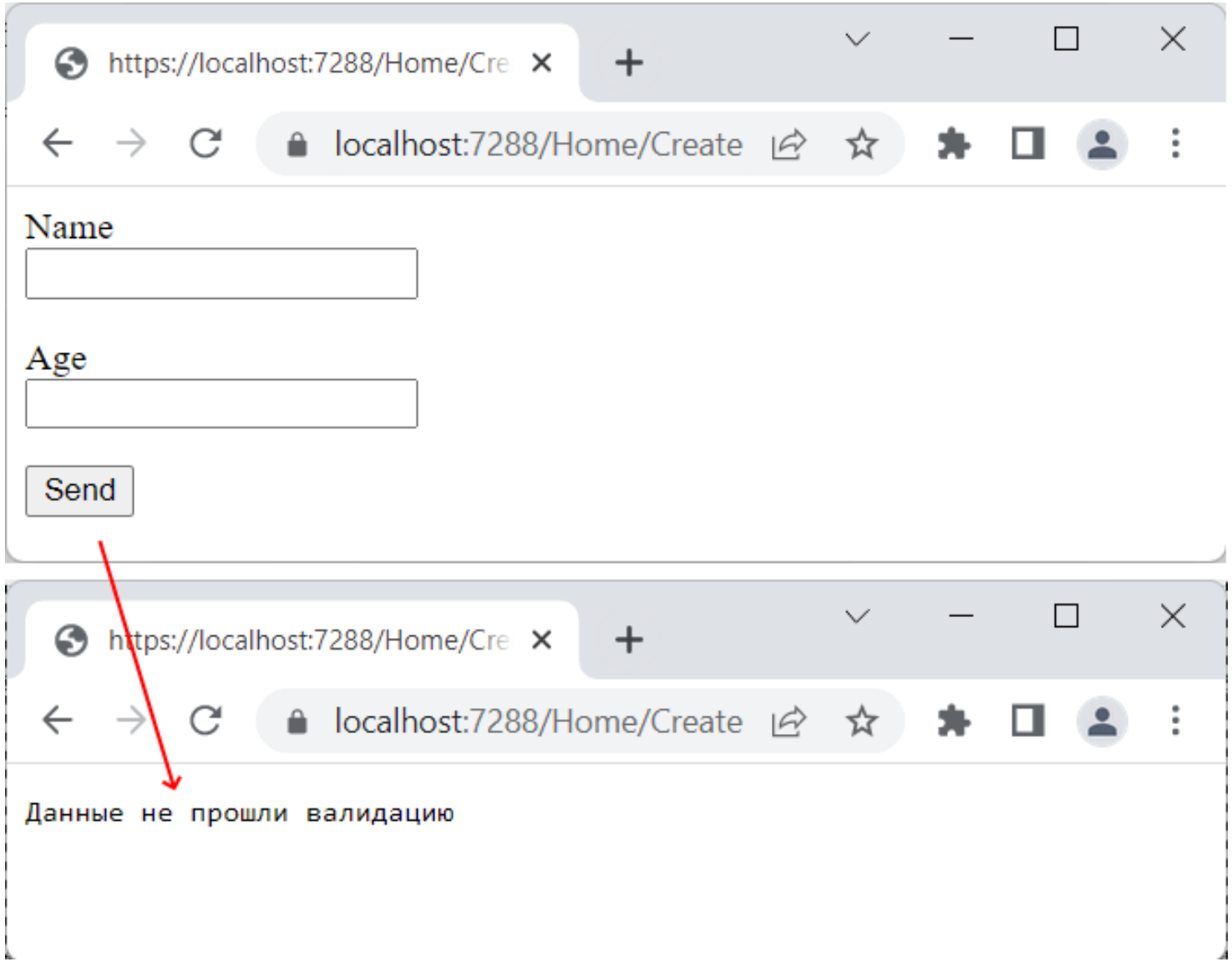
Post-метод Create получает введенные данные в виде модели Person и проверяет их корректность.

Валидация на стороне сервера, то есть в контроллере, осуществляется посредством помощью проверки свойства **ModelState.IsValid**. Объект ModelState сохраняет все значения, которые пользователь ввел для свойств модели, а также все ошибки, связанные с каждым свойством и с моделью в целом. Если в объекте ModelState имеются какие-нибудь ошибки, то свойство ModelState.IsValid возвратит **false**.

Теперь для метода Create в проекте в папке **Home/Create** определим представление **Create.cshtml**, которое будет содержать форму для ввода данных:

```
<form method="post">
    <div>
        <p>
            <label for="name">Name</label><br />
            <input type="text" id="name" name="name" />
        </p>
        <p>
            <label for="age">Age</label><br />
            <input id="age" name="age" />
        </p>
        <p>
            <input type="submit" value="Send"  />
        </p>
    </div></form>
```

Теперь запустим приложение и обратимся к методу Create. И ничего не вводя в поля ввода, нажмем на кнопку отправки:



Валидация на стороне клиента

Кроме валидации на стороне сервера с помощью свойства ModelState в ASP.NET Core MVC можно применять валидацию на стороне клиента, то есть на веб-странице. Валидация на стороне клиента позволяет уменьшить количество обращений к серверу и произвести все действия по проверке значений непосредственно при вводе данных.

Для определения полей для вывода ошибок валидации применяются специальные хелперы. Рассмотрим их применение на примере следующей модели Person:

```
public class Person
{
    [Required(ErrorMessage = "Не указано имя")]
    public string? Name { get; set; }

    [Required(ErrorMessage = "Не указан электронный адрес")]
    public string? Email { get; set; }

    [Required(ErrorMessage = "Не указан возраст")]
    [Range(1, 100)]
    public int Age { get; set; }
}
```

Для валидации на стороне клиента применяется класс **ValidationMessageTagHelper**. Данный tag-хелпер определяется с помощью применения к элементу **** атрибута **asp-validation-for**:

```
<span asp-validation-for="имя_свойства_модели"></span>
```

Атрибут asp-validation-for в качестве значения принимает название свойства модели, для которого будет выводиться сообщение об ошибке валидации. Соответственно для каждого поля ввода мы можем предусмотреть подобный хелпер для вывода ошибок валидации.

Например, форма для ввода значений для выше определенной модели **Person**:

```
@model MvcApp.Models.Person
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers

<form method="post">
    <div>
        <p>
            <label asp-for="Name">Name</label><br />
            <input type="text" asp-for="Name" />
            <span asp-validation-for="Name"  />
        </p>
        <p>
            <label asp-for="Email">Email</label><br />
            <input type="text" asp-for="Email" />
            <span asp-validation-for="Email"  />
        </p>
```

```
        <p>
            <label asp-for="Age">Age</label><br />
            <input asp-for="Age" />
            <span asp-validation-for="Age" />
        </p>
        <p>
            <input type="submit" value="Send"  />
        </p>
    </div>
</form>
```

Чтобы валидация работала нужно подключить скрипты внизу представления.

```
<script src="https://ajax.aspnetcdn.com/ajax/jQuery/jquery-3.5.1.min.js"></script>
<script src="https://ajax.aspnetcdn.com/ajax/jquery.validate/1.17.0/jquery.validate.min.js"></script>
<script src="https://ajax.aspnetcdn.com/ajax/jquery.validation.unobtrusive/3.2.10/jquery.validate.unobtrusive.min.js"></script>
```

Результат работы программы при пустом вводе:

Name

Не указано имя

Email

Не указан электронный адрес

Age

Не указан возраст

Send

Стилизация сообщений об ошибках

Когда происходит валидация, то при отображении ошибок соответствующим полям присваиваются определенные классы css:

- для блока ошибок, который генерируется хелпером ValidationSummaryTagHelper, при наличии ошибок устанавливается класс **validation-summary-errors**.
- для элемента ****, который отображает ошибку для каждого отдельного поля, при наличии ошибок устанавливается класс **field-validation-error**. Если ошибок нет, то данный элемент имеет класс **field-validation-valid.**
- для поля ввода при наличии ошибок устанавливается класс **input-validation-error**. Если ошибок нет, то устанавливается класс **valid.**

Используя эти классы, мы можем настроить отображение сообщений.

Например, изменим представление следующим образом:

```
@model MvcApp.Models.Person
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
<style>
   .field-validation-error {
       color: #b94a48;
   }
   input.input-validation-error {
       border: 1px solid #b94a48;
   }
   input.valid {
       border: 1px solid #16a085;
   }
   .validation-summary-errors {
       color: #b94a48;
   }
</style>
```

tag-хелпер - ValidationSummaryTagHelper применяется для отображения сводки ошибок валидации. Он применяется к элементу ** ** в виде атрибута **asp-validation-summary**

```
<form method="post">
    <div asp-validation-summary="ModelOnly"></div>


</form>
```

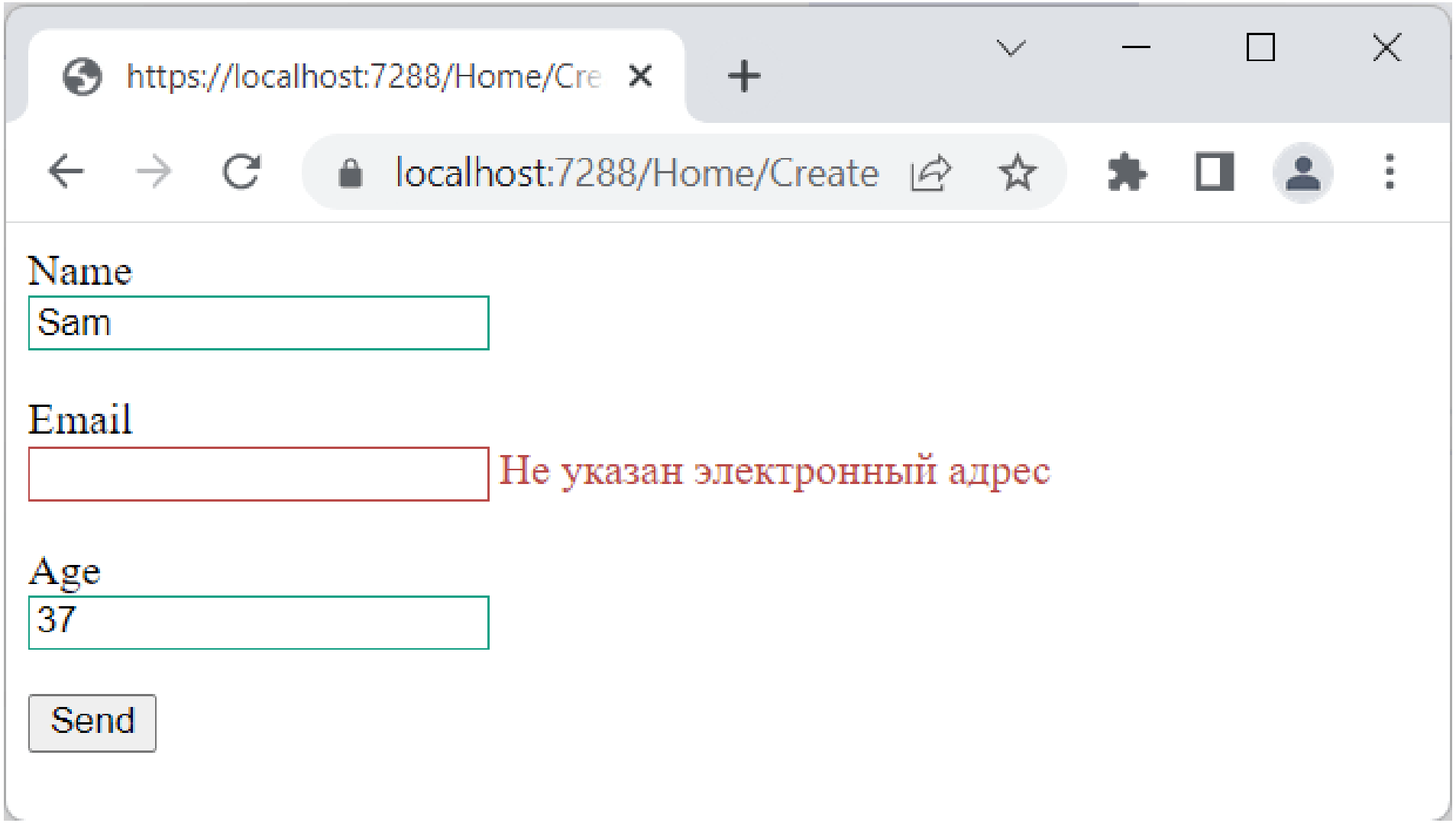
None: ошибки валидации не отображаются

ModelOnly: отображаются только ошибка валидации уровня модели, ошибки валидации для отдельных свойств не отображаются

All: отображаются все ошибки валидации

```
<p>
    <label asp-for="Name">Name</label><br />
    <input type="text" asp-for="Name" />
    <span asp-validation-for="Name"  />
</p>
<p>
    <label asp-for="Email">Email</label><br />
    <input type="text" asp-for="Email" />
    <span asp-validation-for="Email"  />
</p>
<p>
    <label asp-for="Age">Age</label><br />
    <input asp-for="Age" />
    <span asp-validation-for="Age" />
</p>
<p>
    <input type="submit" value="Send"  />
</p>
```

Результат работы программы:



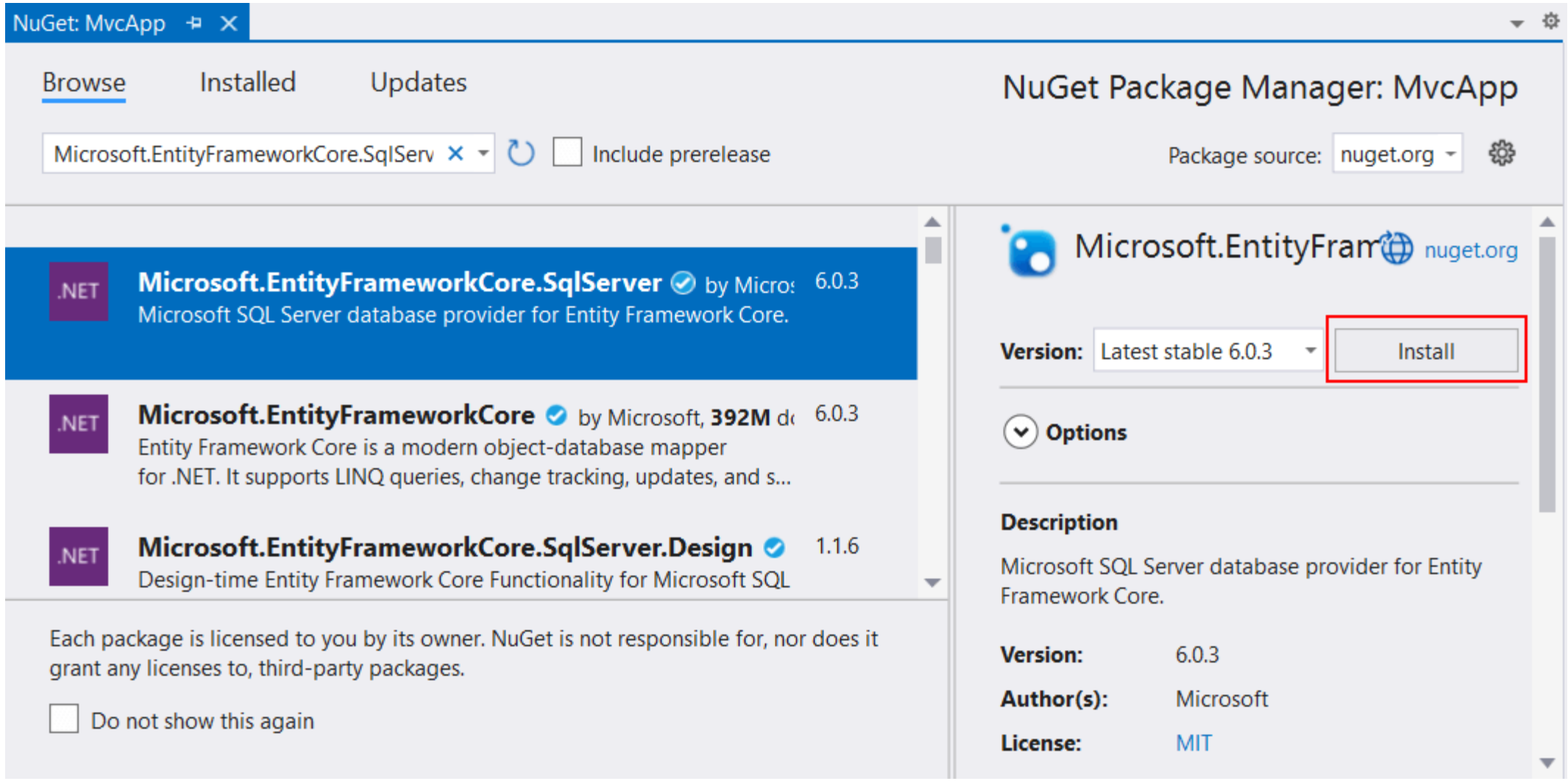
Работа с данными в Entity Framework

Entity Framework представляет прекрасное ORM-решение, которое позволяет автоматически связать обычные классы языка **C#** с таблицами в базе данных. **Entity Framework Core** поддерживает различные СУБД, но в данном случае мы будем работать с базами данных **MS SQL Server**.

Для работы с **Entity Framework** вначале создадим новый проект **ASP.NET Core** по шаблону **ASP.NET Core Empty**.

Для взаимодействия с **MS SQL Server** через **Entity Framework** необходим пакет **Microsoft.EntityFrameworkCore.SqlServer**.

По умолчанию он отсутствует в проекте, поэтому его надо добавить, например, через пакетный менеджер Nuget:



Далее создадим в проекте папку **Models** и в нее добавим новый класс, который назовем **User**:

```
public class User
{
    public int Id { get; set; }
    public string? Name { get; set; } // имя пользователя
    public int Age { get; set; } // возраст пользователя
}
```

Эта модель представляет те объекты, которые будут храниться в базе данных.

Чтобы взаимодействовать с базой данных через Entity Framework нам нужен контекст данных - класс, унаследованный от класса **Microsoft.EntityFrameworkCore.DbContext**.

```
public class ApplicationContext : DbContext
{
    public DbSet<User> Users { get; set; } = null!;

    public ApplicationContext(
            DbContextOptions<ApplicationContext> options)
        : base(options)
    {
        Database.EnsureCreated();
                    // создаем базу данных при первом обращении
    }
}
```

```
public DbSet<User> Users { get; set; } = null!;
```

Свойство **DbSet** представляет собой коллекцию объектов, которая сопоставляется с определенной таблицей в базе данных. При этом по умолчанию название свойства должно соответствовать множественному числу названию модели в соответствии с правилами английского языка. То есть **User** - название класса модели представляет единственное число, а **Users** - множественное число.

Чтобы подключаться к базе данных, нам надо задать параметры подключения. Для этого изменим файл **appsettings.json**, добавив в него определение строки подключения:

```
{
    "ConnectionStrings": {
      "DefaultConnection":
              "Server=(localdb)\\mssqllocaldb;
              Database=usersdb;
              Trusted_Connection=True;"
    },
    // остальное содержимое файла
  }
```

В данном случае мы будем использовать упрощенный движок базы данных **LocalDB**, который представляет легковесную версию **SQL Server Express**, предназначенную специально для разработки приложений.

И последним шагом в настройке проекта является изменение файла **Program.cs**:

```
using Microsoft.EntityFrameworkCore;
using MvcApp.Models; // пространство имен класса ApplicationContext

var builder = WebApplication.CreateBuilder(args);
```

```
// получаем строку подключения из файла конфигурации
string connection =
   builder.Configuration.GetConnectionString("DefaultConnection");

// добавляем контекст ApplicationContext в качестве сервиса в
приложение
builder.Services.AddDbContext<ApplicationContext>(options =>
options.UseSqlServer(connection));

builder.Services.AddControllersWithViews();

var app = builder.Build();

app.MapDefaultControllerRoute();

app.Run();
```

Получение контекста данных в контроллере

Поскольку выше в приложении контекст данных добавляется в виде сервиса, то в конструкторе контроллера мы можем получить переданный контекст данных. Например, пусть для хранения контроллеров в проекте определена папка **Controllers**, в которой есть класс контроллера - **HomeController**:

```
public class HomeController : Controller
{
    ApplicationContext db;
    public HomeController(ApplicationContext context)
    {
        db = context;
    }
}
```

Для взаимодействия с базой данных в контроллере определяется переменная контекст данных **ApplicationContext db**. Причем поскольку в приложении контекст данных добавляется в виде сервиса, то в конструкторе контроллера мы можем получить переданный контекст данных.

Вывод данных

Определим в контроллере три метода, которые будут добавлять новый объект в базу данных и выводить из нее все объекты:

```
public async Task<IActionResult> Index()
{
    return View(await db.Users.ToListAsync());
}
```

В методе Index() с помощью вызова db.Users.ToListAsnc() мы будем получать объекты из бд, создавать из них список и передавать в представление.

Добавление данных

```
public IActionResult Create()
{
    return View();
}
[HttpPost]
public async Task<IActionResult> Create(User user)
{
    db.Users.Add(user);
    await db.SaveChangesAsync();
    return RedirectToAction("Index");
}
```

А в post-методе **Create()** при помощи вызова **db.Users.Add()** для данных из объекта user формируется sql-выражение **INSERT**, а вызов **db.SaveChangesAsync()** выполняет это выражение, тем самым добавляя данные в базу данных.

Далее для хранения представлений добавим в проект папку **Views**, в нее - папку **Home**. Затем в папку **Views/Home** добавим новое представление **Create.cshtml**:

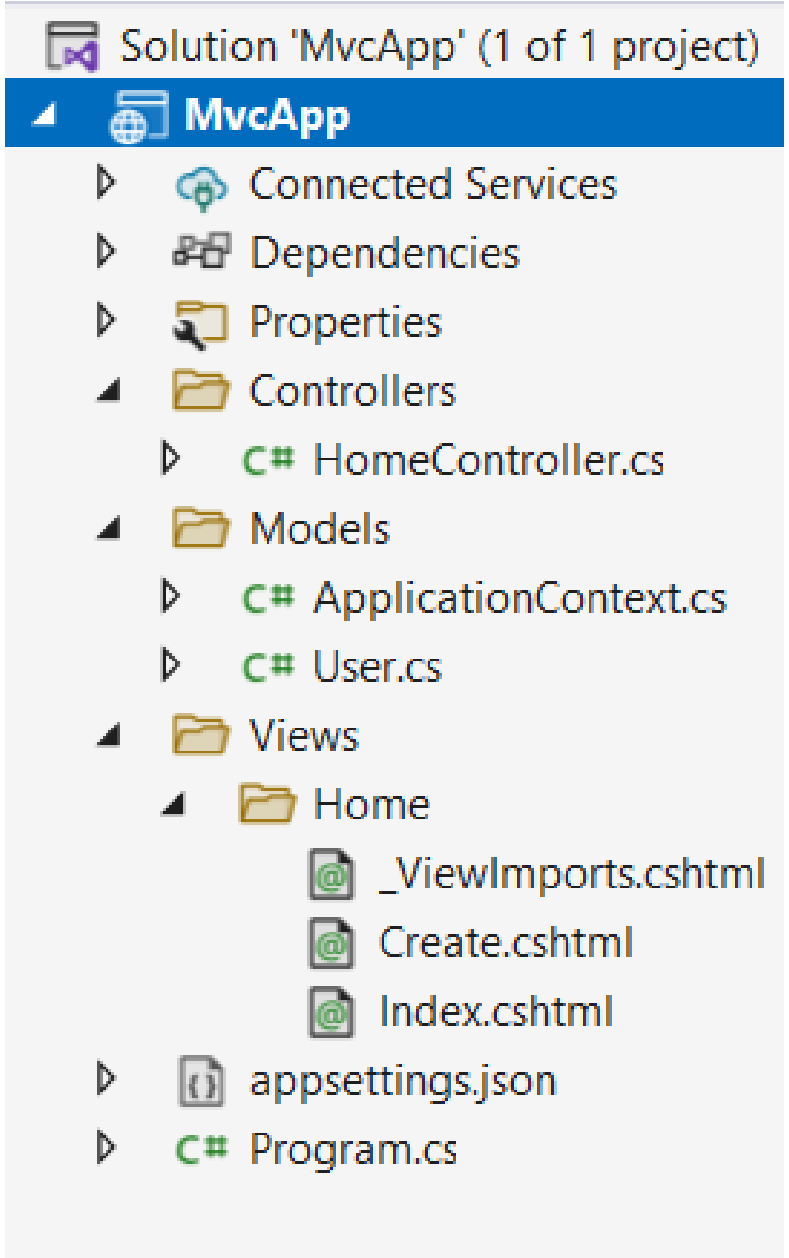
```
@model MvcApp.Models.User
<h2>Добавление пользователя</h2>
<form asp-action="create" asp-controller="home">
    <p>
        <label asp-for="Name">Имя</label><br />
        <input type="text" asp-for="Name" />
    </p>
    <p>
        <label asp-for="Age">Возраст</label><br />
        <input type="number" asp-for="Age" />
    </p>
    <p>
        <input type="submit" value="Отправить" />
    </p></form>
```

Добавим в папку **Views/Home** представление **Index.cshtml**, которое будет отвечать за вывод объектов:

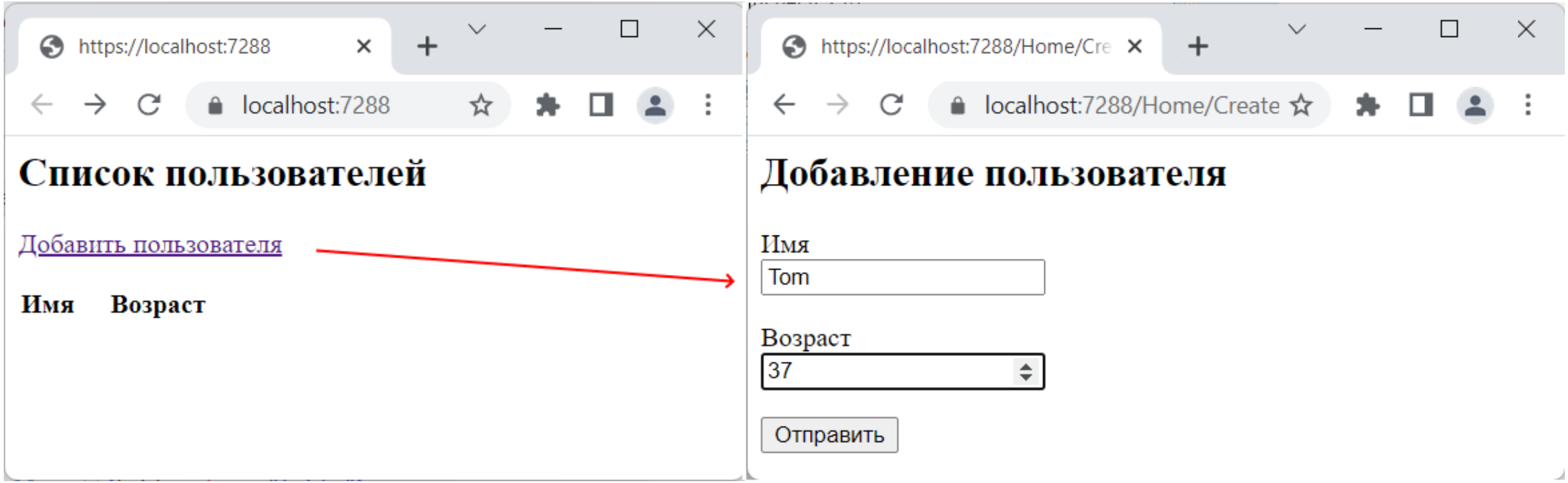
```
@model IEnumerable<MvcApp.Models.User>

<h2>Список пользователей</h2>
<p><a asp-action="Create">Добавить пользователя</a></p>
<table class="table">
     <tr><th>Имя</th><th>Возраст</th></tr>
    @foreach (var item in Model)
    {
        <tr>
            <td>@item.Name</td>
            <td>@item.Age</td>
        </tr>
    }
</table>
```

И также добавим в папку **Views/Home** новое представление **_ViewImports.cshtml**:

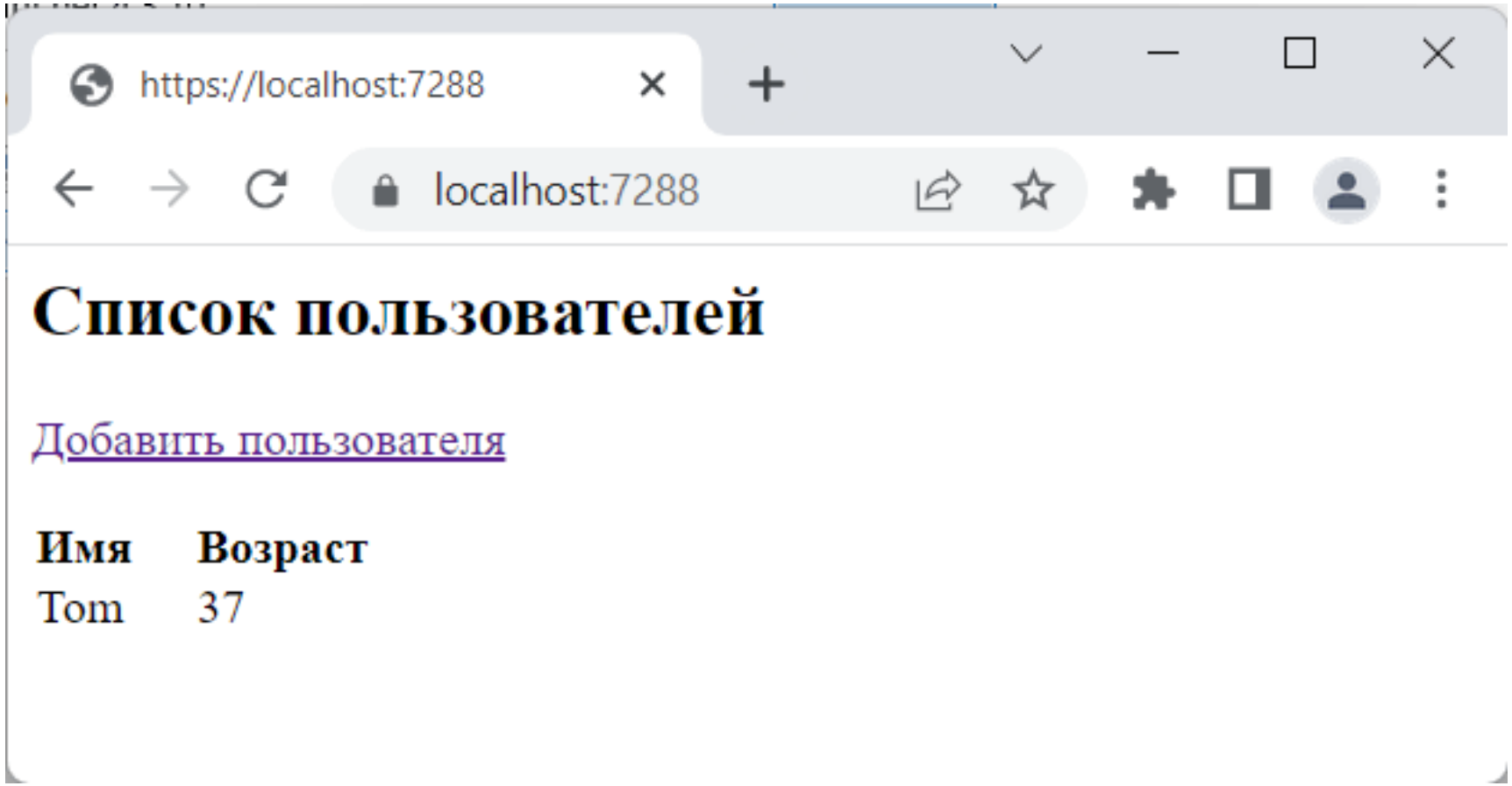
```
@addTagHelper *,
Microsoft.AspNetCore.Mvc.TagHelpers
```



Запустим приложение и обратимся к методу Create:



Введем в форму какие-либо данные и нажмем на копку:



Редактирование и удаление данных

добавим в него возможность редактирования и удаления данных.

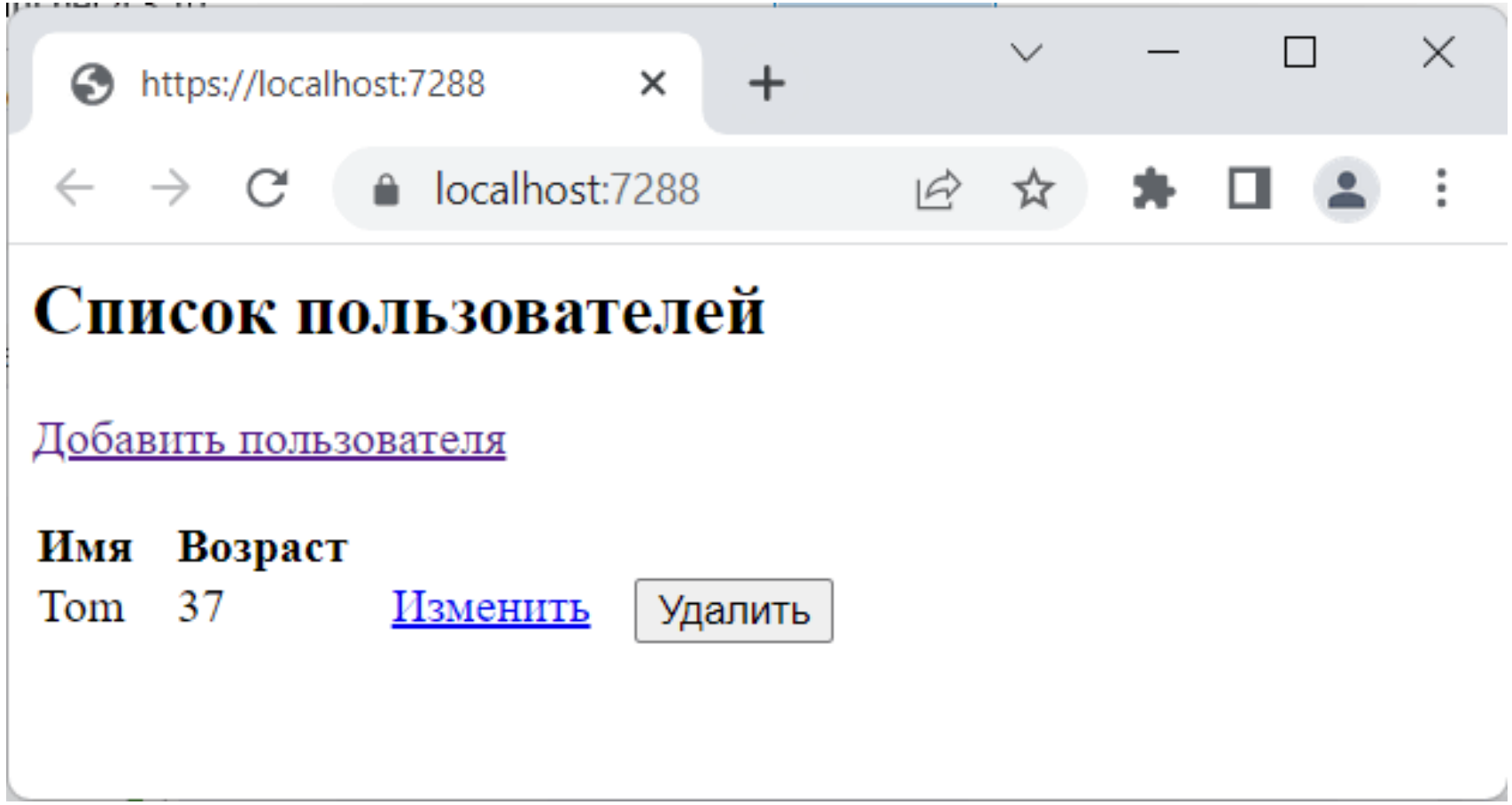
Изменим представление **Index.cshtml**, добавив в него ссылки:

```
@model IEnumerable<MvcApp.Models.User>

<h2>Список пользователей</h2>
<p><a asp-action="Create">Добавить пользователя</a></p>
<table class="table">
     <tr><th>Имя</th><th>Возраст</th><th></th><th></th></tr>
    @foreach (var item in Model)
    {
        <tr>
           @//Следующий слайд
        </tr>
    }
</table>
```

Редактирование и удаление данных

```
<td>@item.Name</td>
<td>@item.Age</td>
<td>
    <a asp-action="Edit" asp-controller="Home" asp-route-
id="@item.Id">Изменить</a>
</td>
<td>
    <form
        asp-controller="Home"
        asp-action="Delete"
        method="post"
        asp-route-id="@item.Id">

        <input type="submit" value="Удалить" />
    </form>
</td>
```

В таблице кроме вывода имени и возраста каждого пользователя также предусмотрена ссылка на редактирование данных, а также форма с кнопкой на удаление данных. Теперь создадим для каждого действия методы.



Удаление

Для создания инфраструктуры удаления добавим в контроллер следующий метод:

```
[HttpPost]
public async Task<IActionResult> Delete(int? id) {
    if (id != null) {
        User? user = await db.Users.FirstOrDefaultAsync(p =>
                                                        p.Id == id);
        if (user != null)
        {
            db.Users.Remove(user);
            await db.SaveChangesAsync();
            return RedirectToAction("Index");
        }
    }
    return NotFound();
}
```

Метод принимает параметр **id**, с помощью которого получаем удаляемый объект из БД и, если он существует, удаляем его с помощью метода **db.Users.Remove**(). Данный метод генерирует **sql-выражение DELETE**, которое выполняется вызовом **db.SaveChangesAsync**().

Стоит отметить, что данный метод **Delete** обрабатывает только запросы типа **POST**. Почему? Дело в том, что использование **get-методов** не безопасно. Например, нам могут прислать письмо с картинкой:

```
<img src="http://адрес_нашего_сайта/Home/Delete/1" />
```

```html
<img src="http://адрес_нашего_сайта/Home/Delete/1" />
```

И при открытии письма на сервер будет отправлен **get-запрос**. И если бы метод **Delete** обрабатывал бы **get-запросы**, то объект с id=1 был бы удален из базы данных.

Именно поэтому данный метод обрабатывает запросы **POST**, а для обращения к этому методу в представлении определена не просто ссылка, а форма с кнопкой, по нажатию на которую выполняется **post-запрос**.

И еще надо отметить, что в post-методе **Delete** мы можем произвести небольшую оптимизацию. Иногда бывает важно узнать перед удалением, а есть ли такой объект в БД. Однако в данном случае мы получаем два запроса к бд - один на получение объекта и второй на его удаление. И мы можем оптимизировать метод следующим образом:

```
[HttpPost]
public async Task<IActionResult> Delete(int? id){
    if (id != null)
    {
        User user = new User { Id = id.Value };
        db.Entry(user).State = EntityState.Deleted;
        await db.SaveChangesAsync();
        return RedirectToAction("Index");
    }
    return NotFound();
}
```

```
[HttpPost]
public async Task<IActionResult> Delete(int? id){
    if (id != null)
    {
        User user = new User { Id = id.Value };
        db.Entry(user).State = EntityState.Deleted;
        await db.SaveChangesAsync();
        return RedirectToAction("Index");
    }
    return NotFound();
}
```

В данном случае нам важен только id объекта. Поэтому создаем новый объект user и устанавливаем у него состояние Deleted: db.Entry(user).State = EntityState.Deleted;. Это выражение опять же сгенерирует sql-выражение DELETE.

Редактирование

Для редактирования объекта добавим в контроллер следующую пару методов:

```
public async Task<IActionResult> Edit(int? id)
{
    if(id!=null)
    {
        User? user = await
                     db.Users.FirstOrDefaultAsync(p => p.Id == id);
        if (user != null)
           return View(user);
    }
    return NotFound();
}
```

Редактирование

```
[HttpPost]
public async Task<IActionResult> Edit(User user)
{
    db.Users.Update(user);
    await db.SaveChangesAsync();
    return RedirectToAction("Index");
}
```

GET-версия метода **Edit** возвращает форму с данными объекта, которые пользователь может отредактировать. А **post-версия Edit** получает отредактированные данные в виде объекта **user** и с помощью метода **db.Users.Update(user**) для этих данных будет генерироваться **sql-выражение UPDATE**, которое будет выполнено вызовом **db.SaveChangesAsync**()

И также добавим для этих методов в папку **Views/Home** представление **Edit.cshtml**:

```
@model MvcApp.Models.User

<h2>Редактирование пользователя</h2>
<form
   asp-action="edit"
   asp-controller="home"
   asp-route-id="@Model.Id">

   @//След. слайд

</form>
```

```
<p>
    <label asp-for="Name">Имя</label><br />
    <input type="text" asp-for="Name" />
</p>
<p>
    <label asp-for="Age">Возраст</label><br />
    <input type="number" asp-for="Age" />
</p>
<p>
    <input type="submit" value="Сохранить" />
</p>
```

Установка атрибута asp-route-id="@Model.Id" позволяет при отправке данных вместе с ними также отправить и значение Model.Id. Без этого значения Entity Framework не сможет обновить соответствующую модель в базе данных.